Social Login via OpenID Connect (Google / Facebook)

Contents

Social Login (Google / Facebook) **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 3
- Related . . . 3

Social Login (Google / Facebook)

Scope

Third-party sign-in and account linking. Local identity rules are in `../../40-security/authentication-authorization/ST`

Official requirements

- Subject V.1: registration by social account (Facebook or Google), and linking a social account to an existing account.
- Subject V.5: the mobile app must support authentication through a social network.
- **OpenID Connect Core 1.0** on top of OAuth 2.0 (RFC 6749), with OAuth 2.0 Security Best Current Practice (RFC 9700), and each provider's own documented flow.
- OWASP ASVS 5.0.0 chapter **V10 OAuth and OIDC** at Level 2 — see `../../../owasp/ASVS-CONFORMANCE.md`.

Project policy

Protocol: OIDC, not bare OAuth 2.0

OAuth 2.0 is an **authorisation** protocol. An access token says what the bearer may do, not who they are, and it may have been minted for a different application. Identity comes from a validated **ID token** only. Decision: `brain/decisions/ADR-0008-oidc-social-login.md`.

Flow

- Native apps use the **authorization code flow with PKCE**. The implicit flow is forbidden; RFC 9700 deprecates it.
- The client secret lives on the server only. A mobile app is a public client and cannot keep a secret.
- `state` and `nonce` are generated server-side, single-use, and verified.
- Redirect URIs are registered exactly and allow-listed. No wildcard, no open redirect.
- After validation the server issues **our own** tokens. A provider token is never accepted as an API credential.

ID token validation — every check, server-side, in this order

1. Signature verified against the provider's JWKS, fetched from its discovery document and cached with key rotation in mind.
2. `iss` matches the expected issuer exactly.
3. `aud` matches our client id. **This is the check that stops token substitution** — a token minted for another application must be rejected.
4. `exp` and `iat` within tolerance.
5. `nonce` matches the one this server generated for this authentication.
6. `email_verified` is `true` before the email is used to match an existing account.

A profile object sent by the client is never trusted, at any step.

Provider differences — do not assume symmetry

Provider	Reality
Google	full OpenID Connect. Discovery at `https://accounts.google.com/.well-known/openid-configurat` ID token validation is the documented path.
Facebook	**not uniformly OIDC.** Classic Facebook Login is OAuth 2.0 plus the Graph API; Facebook *Limited Login* issues an OIDC ID token. The integration branches: OIDC validation where an ID token exists, server-side token inspection against Facebook's own endpoint where it does not.

Treating the two providers as identical produces either a broken integration or an insecure one.

Account model

- A provider identity is `(provider, providerUserId)` and maps to at most one local user.
- Provider email is used to match an existing account **only when the provider asserts the email is verified**. Otherwise create a new account or require explicit linking. Unverified-email matching is an account-takeover path.
- Linking requires an already-authenticated session plus a fresh provider authentication.
- A user must always retain at least one usable sign-in method. Unlinking the last one is refused unless a password is set.
- Store the provider identifier and minimal profile. Do not store provider access tokens unless a feature needs them; if stored, encrypt at rest and keep the scope minimal.

Scopes

Request the minimum: identity and email. No contacts, no posting permissions, no extra scopes "for later".

Checklist

- ☐ Authorization code + PKCE, no implicit flow
- ☐ Identity taken from the **ID token**, never from an access token
- ☐ ID token verified server-side against JWKS, including `aud` and `nonce`
- ☐ Facebook path handled explicitly, not assumed identical to Google
- ☐ `state` and `nonce` generated and verified server-side
- ☐ Email matching only on a provider-verified email
- ☐ Client secrets absent from the mobile bundle
- ☐ Minimum scopes requested
- ☐ User cannot be locked out by unlinking their last method

References

- RFC 6749 OAuth 2.0 — https://www.rfc-editor.org/rfc/rfc6749
- RFC 9700 OAuth 2.0 Security BCP — https://www.rfc-editor.org/rfc/rfc9700
- OpenID Connect Core — https://openid.net/specs/openid-connect-core-1_0.html
- OpenID Connect Discovery — https://openid.net/specs/openid-connect-discovery-1_0.html
- OWASP OAuth 2.0 Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html
- Google Identity — https://developers.google.com/identity
- Facebook Login — https://developers.facebook.com/docs/facebook-login/

Related

`../../40-security/authentication-authorization/STANDARD.md` · `../../40-security/secrets-management/STANDA`